← Back to blog

Scaling Fastly Network: Fighting FIB | Fastly

João Taveira Araújo
Director of Engineering, Fastly

May 11, 2016

Product | Performance | Edge network

This post is the first in a series detailing the evolution of network software at Fastly. We're unique amongst our peers in that from inception, we've always viewed networking as an integral part of our product rather than a cost center. We rarely share what we do with the wider networking community however, in part because we borrow far more from classic systems theory than contemporary networking practice.

Building abstractions

Before delving further, it is important to disambiguate that while we write software for networks, we shy away from using the term software-defined networking (SDN) to describe what we do.

For one, SDN perpetuates the misconception that computer networking was ever about anything other than software — "running code and rough consensus" was an IETF mantra long before networking vendors reduced it to a sound bite. The term itself has since been adopted and reinterpreted in order to legitimize specific approaches to network management. In the process, SDN has evolved into the most effective of buzzwords: a portmanteau so devoid of meaning you begin to question your own understanding of the problem.

The fact that the wider networking industry is rediscovering the relevance of software is inevitable given the sheer scale of modern networks. With the successive waves of virtualization and then containerization, it's not uncommon for a single data center rack to contain more uniquely addressable endpoints than the entire internet had just three decades ago. The complexity of managing so many devices has understandably shifted much of the community's attention towards the need for automation in order to reduce spiralling costs in operating infrastructure.

While this is an important development, automation is not our end goal. Our primary focus is in uncovering effective abstractions for building network applications.

If the delineation between both feels subtle, consider the challenge of propagating the mapping between hostnames and addresses to all nodes in a network. There are numerous ways one could automate this process — from something as simple as a cron job to emailing nodes with updates1. Instead, the solution adopted by early internet pioneers was the Domain Name System (DNS), a hierarchical decentralized naming system. Over time, the layer of indirection provided by DNS has been crucial to the development of a number of value-added services such as content delivery networks. Automation saves time, abstractions make time work for you.

Scaling out

The expertise of our core founding team lay primarily in high-performance systems, which turns out to be essential to offering low latency, distributed caching as a service. Bootstrapping a content delivery network however is inherently tricky, given incumbents start with a larger geographic footprint. Our initial offering therefore focused on what our industry had failed to provide — unprecedented visibility and control over how content gets delivered at the edge.

During our formative years our lack of experience in networking was largely irrelevant — our typical point of presence (POP) was composed of two hosts directly connected to providers over Border Gateway Protocol (BGP). By early 2013 we had grown enough that this number of hosts was no longer enough. Scaling our topology by connecting more caches directly to our providers, as shown in figure 1a, was not an option. Providers are reluctant to support this due to the cost of ports and the configuration complexity of setting additional BGP sessions.

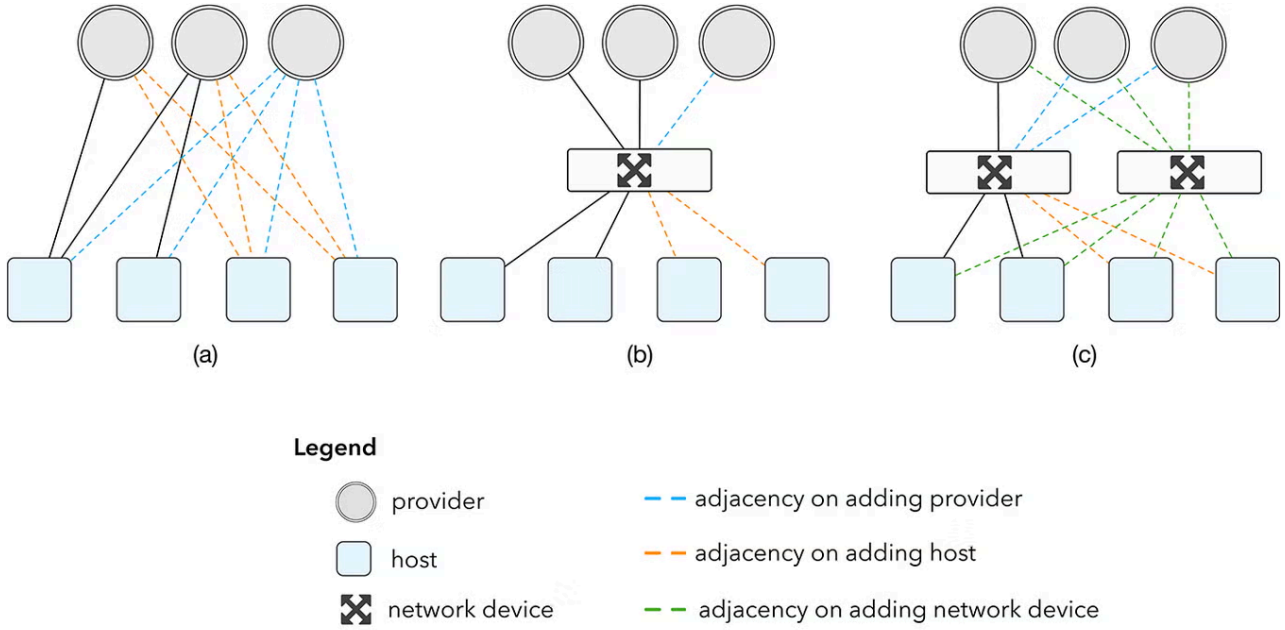

Figure 1: impact of scaling network topology

An obvious solution to this problem would be to install a network device, as shown in figure 1b, which neatly decouples the increase in number of devices from the increase in number of providers. This network device would typically be a router, which is a highly specialized device for forwarding traffic with a price tag to match. While this would be an acceptable compromise if the overall volume of devices were low, the nature of a content delivery network is to constantly expand both in geographic reach and volume of traffic. Today, our smallest POP has at least two network devices, as shown in figure 1c.

An overview of how such a network would work with a router is shown in figure 2. A router receives routes directly from providers over BGP, and inserts them into the Forwarding Information Base (FIB), the lookup table implemented in hardware used for route selection. Hosts then forward traffic to the router, which forwards packets to the appropriate next hop according to the resulting lookup in the device FIB.

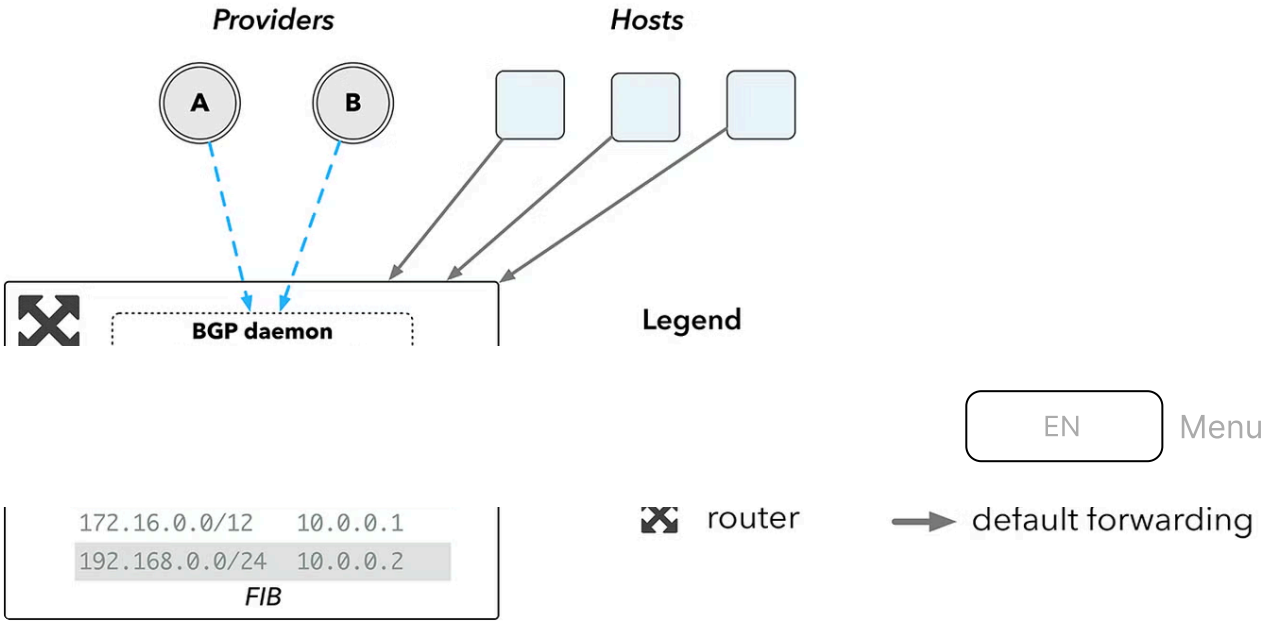

Figure 2: network topology using a router

The larger the FIB, the more routes a device can hold. Unfortunately, the relationship between FIB size and cost is not linear. Border routers must be able to hold the full internet routing table, which now exceeds over 600,000 entries2. The hardware required to support this space is the primary cost associated with routers.

Routing without routers

In traditional cloud computing environments, the cost of border routers is quickly dwarfed by the sheer volume of servers and switches they are intended to serve. For CDNs however, the cost is much more than mere inconvenience. In order to place content closer to end users, CDNs must have a large number of vantage points from which they serve content. As a result, network devices can represent a significant amount of the total cost of infrastructure.

The idea of dropping several millions of dollars on overly expensive networking hardware wasn't particularly appealing to us. As systems engineers we'd much rather invest the money in commodity server hardware, which directly impacts how efficiently we can deliver content.

Our first observation was that we didn't need most of the features provided by routers, because we were not planning on becoming a telco any time soon. Switches seemed like a much more attractive proposition, but lacked the one feature that made routers useful to us in the first place: FIB space. At the time, switches could typically only hold tens of thousands of routes in FIB, which is orders of magnitude less than we needed. By 2013, hardware vendors such as Arista had begun to provide a feature that could overcome this physical limitation: they would allow us to run our own software on switches.

Freed from the shackles of having to obey the etiquette of sensible network design, our workaround took form relatively quickly. Instead of relying on FIB space in a network device, we could push routes out towards the hosts themselves. BGP sessions from our providers would still be terminated at the switch, but from there the routes would be reflected down to the hosts.

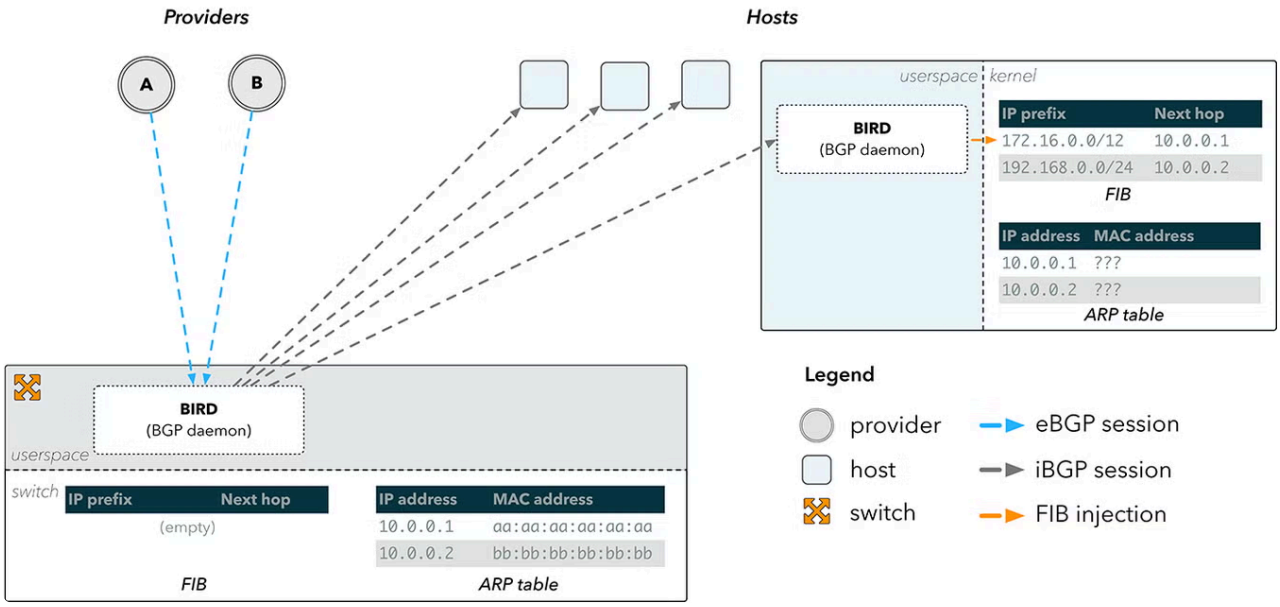

Figure 3: BGP route reflection

This approach is presented in figure 3. An external BGP (eBGP) session is terminated in a userspace BGP daemon, such as BIRD, which runs on our switch. The routes received are then pushed over internal BGP (iBGP) sessions down to a BIRD instance running on the hosts, which then injects routes directly into the host kernel.

This solves our immediate problem of bypassing the switch FIB entirely, but it doesn't entirely solve the problem of how to send packets back towards the internet. A FIB entry is composed of a destination prefix (where a packet is going) and a nexthop address (where it's going through). In order to forward a packet to a nexthop, a device must know the nexthop's physical address on the network. This mapping is stored in the Address Resolution Protocol (ARP) table.

Figure 3 illustrates that the switch has the appropriate ARP information for our providers, since it is directly connected to them. The hosts however do not, and therefore cannot resolve any of the nexthops they have been provided over BGP.

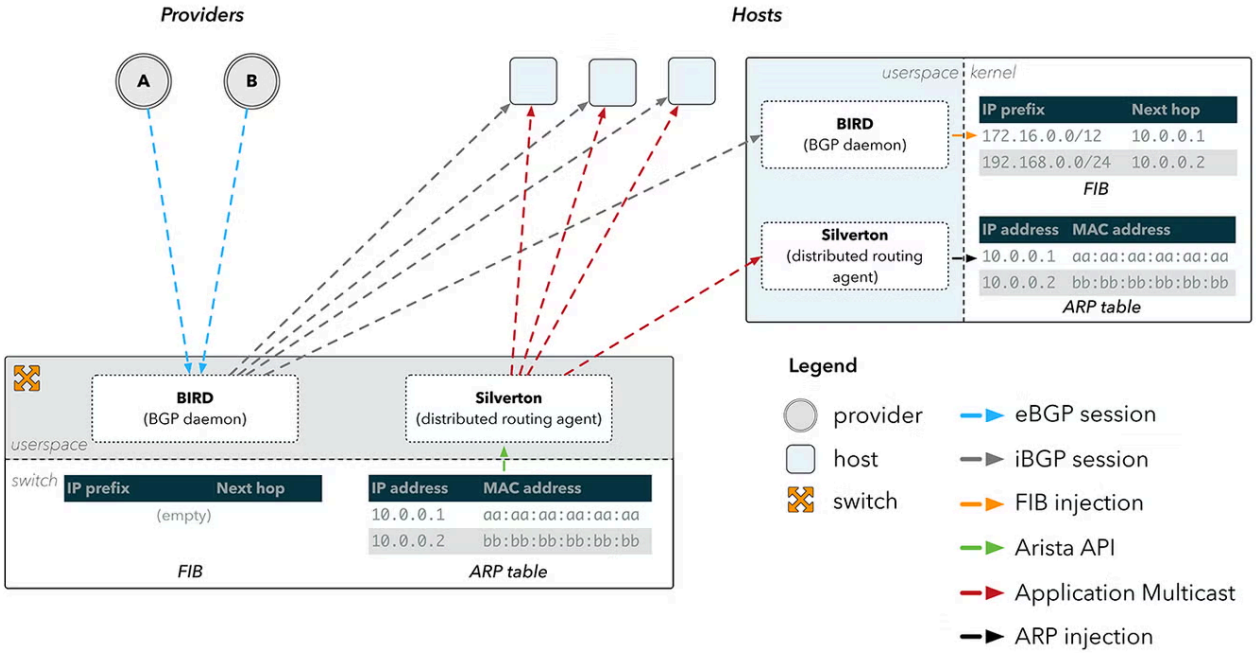

Figure 4: ARP propagation using Silverton

Silverton: a distributed routing agent

This was the starting point for Silverton, our custom network controller which orchestrates route configuration within our POPs. We realized that we could simply run a daemon on the switch which subscribed to changes to the ARP table through the API provided on Arista devices. Upon detecting a change to a provider's physical MAC address, Silverton could then disseminate this information throughout the network, and clients on the hosts would reconfigure our servers with information on how to directly reach our providers.

For a given provider IP and MAC address, the first step performed by the client-side agent of Silverton is to fool the host into believing the IP is reachable directly over an interface, or link local. This can be achieved by configuring the provider IP as a peer on the interface, and is easily replicated on Linux by using iproute:

```
$ ip addr add <localip> peer 10.0.0.1 dev eth0
```

If the host believes the provider IP is link local, it will be forced to look up the MAC address for that IP in its ARP table. We can manipulate that too:

```
$ ip neigh replace 10.0.0.1 lladdr aa:aa:aa:aa:aa:aa nud perman
```

Now every time a route lookup for a destination returns the nexthop `10.0.0.1`, it will end up sending traffic to `aa:aa:aa:aa:aa:aa` directly. The switch receives data *frames* from the host towards a physical MAC address which is known to be directly connected. It can inspect which interface to forward the frame along by inspecting its local MAC address table, which maintains a mapping between a destination MAC address and the outbound interface.

While this entire process may seem convoluted in order to merely forward packets out of a POP, our first iteration of Silverton contained less than 200 lines of code, and yet instantly saved us hundreds of thousands of dollars for every POP we deployed. Importantly, unlike hardware, software can also be incrementally refined. Over time, Silverton has grown to encompass all of our dynamic network configuration, from labelling description fields to manipulating routing announcements and draining BGP sessions.

More than saving money however, Silverton provided us with a valuable abstraction. It maintained the illusion that every host is directly connected to every provider, which was our starting point (figure 1a). By maintaining multiple routing tables in the kernel and selecting which table to look up on a per-packet basis, we were able to build tools and applications on top of Silverton which can override route selection. An example of this is an internal utility called *st-ping*, which pings a destination over all connected providers:

```
Pinging  <hostname>  via 14 upstreams.
  Upstream   intr          Nexthop  Sent   Loss        Min        Avg        Max        Dev
  cogent                              10   0.0%     13.017     13.051     13.164      0.044
  cogent                              10   0.0%     13.011     13.035     13.091      0.024
  cogent                              10   0.0%     13.008     13.021     13.046      0.052
  cogent                              10   0.0%     13.018     13.037     13.083      0.136
* l3                                  10   0.0%      10.52     10.546     10.621      0.133
* l3                                  10   0.0%     10.523     10.533     10.571      0.138
  ntt                                 10   0.0%     13.952     13.973     14.024      0.131
  ntt                                 10   0.0%     13.949     13.974     14.148      0.131
  ntt                                 10   0.0%      13.95     13.961     13.984      0.149
  ntt                                 10   0.0%     13.946     13.964       14.0      0.159
  telia                               10   0.0%     14.198     14.326     14.963      0.234
  telia                               10   0.0%     14.184     14.406     15.468      0.393
  telia                               10   0.0%     14.379     14.417     14.558      0.152
  telia                               10   0.0%     14.383     14.405     14.444      0.056
```

Figure 5: ping over all transit providers in a POP. Delay is shown for a single destination IP, and therefore not representative of global provider performance.

Pushing path selection all the way to the application allowed us to develop far greater introspection into network behavior, which we then used to drive content delivery performance at the edge.

Going forward: what else can we get away with?

Silverton served as a reminder that nothing is stronger than an idea whose time has come.

Had we attempted to implement Silverton two years earlier, we would have hit a wall: no vendor on the market would have provided us with programmatic access to the core networking components we needed. Fortunately we found ourselves looking for switches at a time when Arista were starting to formalize access to the internal APIs on their devices.

Had we attempted to implement Silverton today, we would have long since bought into the collective delusion that you need routers to do routing. As it turns out, routers are about as expensive to get rid of as they are to buy, since the people you hire to configure them are just as specialized as the hardware they maintain. By avoiding routers entirely we were able to build a networking team with a different mindset on how you operate a network, and we've been reaping the benefits ever since.

Upon validating the first proof of concept for Silverton in early 2013, a natural question arose: *what else can we get away with?* The next post in this series will explore how we applied the same principles of deception in order to handle inbound traffic and perform seamless load balancing.

---

[1] RFC849. Suggestions for improved host table distribution (https://tools.ietf.org/html/rfc849)

[2] Growth of the BGP Table http://bgp.potaroo.net/

Products
Edge Cloud Platform
DDoS Protection

Services
Professional Services
Managed CDN

Learn
Documentation
Developers

Support
Support Center
Network Status

Ready to get started?

Get in touch with us today

Talk to an expert →

Partners

News

Investor Relations

Trust

Fastly with Cloud Providers

X ↗

LinkedIn ↗

Instagram ↗

YouTube ↗

© Fastly 2026

Terms of Service

Privacy policy

Acceptable Use